Writing Living Reference

A student guide for maintaining `features/reference/` — the **current integrated snapshot** of the product on `dev`.

Index: README.md
Files: api.md · data-model.md · behavior.md
Related: framework.md (merge / DoD) · writing feature design · writing feature requirements

---

What living reference is

Living reference answers: *"What does the app look like / enforce right now on `dev`?"*

Artifact	Answers	Authorizes new work?
Feature spec (`feature-N-*.md`)	*What changes* this feature adds	**Yes** — implement from here
Living reference (`features/reference/`)	*What exists* after merges	**No** — snapshot only
ADRs	*Why* cross-cutting architecture	Indirectly (constraints)
Cursor rules	*How* to code	Patterns, not product scope

Feature specs are **deltas**. Reference files are **current state**. After Feature 5 ships, `api.md` shows the full `/todo/` surface; Feature 5's spec only describes the due-date delta.

Reference is **not** auto-generated from specs. Update it in the **same PR** as the implementation (required Definition of Done).

The three files

File	Contents	Update when…
api.md	Routes, auth, request/response shapes, errors	Routes or payloads change
data-model.md	Tables, columns, types/rules, associations	Schema or associations change
behavior.md	Product rules in force (ownership, sort, validation, UI rules)	Rules change (not mere path renames)

Which file(s) for this change?

You changed…	Update
New/changed endpoint or JSON fields	**api.md**
New table/column/FK/association	**data-model.md**
Ownership, sort order, validation limits, empty-state copy, overdue rules, where Log out lives	**behavior.md**
Only internal refactor, same external contract	Usually **none** (confirm no behavior change)
Screen layout polish already implied by existing rules	Usually **none**; if a **stated UI rule** changes → **behavior.md**

Many features touch **two or three** files (e.g. Feature 5: `dueDate` column → data-model + api + overdue rule → behavior).

Principles for writing living reference

1. **Describe now, not history.** Reference is the integrated product. Provenance tables record *which feature introduced* an area — not a full changelog essay.
2. **Never authorize scope.** If someone needs new behavior, they write/update a **feature spec** first. Do not "fix production" by editing only `api.md`.
3. **Update in the same PR as code.** DoD — not a follow-up "docs later" task.
4. **Edit as a delta on current state.** Add a section, field, or row; don't rewrite the whole file unless structure is broken.
5. **Match shipped code.** If the PR returns `dueDate` as `YYYY-MM-DD`, the reference must say that — not an older draft from the spec that never shipped.
6. **Keep Gherkin in features.** Reference may index rules; deep scenarios stay in `feature-N-*.md` Acceptance Criteria.
7. **One fact, one place.** Column types live in **data-model**; HTTP paths in **api**; "lists sort A–Z" in **behavior** — don't triple-paste novels.
8. **Record provenance.** When you add an area, add/update the Feature provenance table (`Introduced` / Feature N).
9. **Align with Accepted ADRs.** Ownership/`404` rules should match ADR-0002; don't invent a conflicting "use 403" in behavior.md.
10. **Stubs stay empty until Feature 1+.** New apps / `reset:example` start with empty reference shells; fill as features merge.

Writing api.md

What belongs

- Base path and auth convention (`Authorization: Bearer …`)
- Endpoint tables: Method, Path, Auth, Purpose
- Request/response JSON for create/update and important errors
- Feature provenance for API areas

Principles

Do	Don't
Document the **integrated** API after this merge	Only paste this feature's API section and delete older routes
Show real status codes and error `{ "message": … }` shapes	Vague "returns an error"
Note Auth Yes/No per route	Assume readers remember Feature 1
Update payloads when fields are added (e.g. `dueDate`)	Leave Feature 3 payloads forever when Feature 5 shipped

Example delta (Feature 5 style)

- Keep existing todo routes.
- Extend create/update body and response docs with optional `dueDate`.
- Add provenance row: Todo `dueDate` → Feature 5.

Writing data-model.md

What belongs

- Each table: Field / Type / Rules

- Associations (`User hasMany List` , cascade deletes, …)
- Feature provenance for schema areas

Principles

Do	Don't
Reflect **shipped** Sequelize/MySQL reality	Speculative columns "we might need"
Mark PK, FK, unique, defaults, DATEONLY vs DATETIME	Copy-paste model `.js` source
For deltas, add/change **only** the new fields and note the feature	Delete unrelated tables from the file
Keep associations in sync with `models/index.js`	Orphan "hasMany" that code doesn't implement

---

Writing behavior.md

What belongs

Product **rules** currently enforced — ownership, sorting, validation limits, session lifetime, empty-state **copy**, overdue styling rules, MenuBar/logout placement.

Use a compact table shape:

Rule	Enforcement	Introduced
Cross-user access → **`404`**, never `403`	Controllers + helpers	ADR-0002; Features 2–4
Lists returned **alphabetically by name**	`findAll` order	Feature 2

Principles

Do	Don't
State the rule in product language	Dump full Gherkin scenarios
Point at enforcement (middleware, helper, UI)	"The app handles this somehow"
Add a row when a **rule** changes	Duplicate every API path from api.md
Group by area (Auth, Ownership, Lists, Todos, …)	One undifferentiated bullet blob

behavior.md vs Screen Requirements: Screen Requirements in the feature authorize UI for that slice. After merge, durable UI **rules** (exact empty-state string, overdue condition) that others must not regress belong in **behavior.md**.

Workflow (per feature PR)

1. Implement from `features/feature-N-*.md` only.
2. Before marking DoD complete, ask:
   - Schema change? → **data-model.md**
   - Route/payload change? → **api.md**
   - Rule change? → **behavior.md**
3. Update provenance tables.
4. Skim for contradictions (api field missing from data-model; behavior sort order ≠ api `order`).

5. List the files you touched in the feature's **Agent implementation request** / DoD checkboxes.

Agent implementation request (reminder)

Features should say which reference files to update, for example:

```
If API routes, payloads, schema, or product rules changed per this spec, update
@features/reference/api.md, @features/reference/data-model.md, and/or
@features/reference/behavior.md in the same PR to match shipped code.
```

Reference updates for this feature: list only the files that will change (or `none`).

Provenance tables

Keep a short "who introduced this" index in README and/or each file:

Area	Introduced
Auth, sessions	Feature 1
Lists CRUD + Dashboard lists view	Feature 2
…	…

Principles: one row per capability area; update when a **new** area appears; for field-level deltas, a row like `Todo dueDate` → Feature 5 is enough.

Drift and repair

Symptom	Action
Code has a route not in api.md	Update **api.md** (if intentional) or remove the route (if unauthorized)
Spec says one thing, reference another, code a third	Spec authorizes; fix code + reference to match the **shipped** spec delta
behavior.md missing a rule that tests enforce	Add the rule row
Tempted to add behavior only in reference	Stop — write a feature delta first

---

Checklist (before merge)

- [ ] Knew which of api / data-model / behavior apply (or consciously chose none)
- [ ] Edits describe **current** integrated state after this PR
- [ ] No new scope that isn't in the feature spec
- [ ] Provenance updated for new areas/fields
- [ ] api ↔ data-model ↔ behavior consistent with each other and with tests
- [ ] Feature DoD / Agent request lists the reference files touched
- [ ] Did not "clean up" by deleting still-shipped older feature surface area

---

Anti-patterns

Avoid	Do instead
Updating reference weeks after merge	Same PR as implementation
Replacing api.md with only this feature's endpoints	Merge delta into the full snapshot
Using reference as the requirements doc	Feature specs authorize; reference reflects
Copying entire feature Data Model / API sections verbatim forever	Integrate into current-state structure; drop redundancy
behavior.md as a second Gherkin suite	Rule index + enforcement pointer
Editing reference to "make the agent happier" without a spec	Spec first (constitution)

How this fits the other guides

```
Author the delta (what/how to build)  → writing-feature-requirements.md
                                        writing-feature-design.md
Ship the delta + snapshot "what is"   → writing-living-reference.md  (this file)
Architecture why                      → docs/adr/writing-adrs.md
Quality bars                          → docs/nfr/writing-quality-attributes.md
```

After merge, the next feature's authors read **reference** to know the baseline and write a **new feature file** for the next delta — they do not rewrite Features 1–N.